

Building and Comparing Segmentation Strategies for Coronary Artery Calcium (CAC)

Table of Contents

1. Applicant Information

- Personal Details (Name, Email, Contact, University, Degree, CGPA)
- Technical Profiles (GitHub, LinkedIn, Portfolio)

2. Abstract

3. Problem Context and Motivation

3.1 Background of the Problem

3.2 Importance in Scientific / ML4SCI Context

4. Proposed Solution and Technical Approach

4.1 System Pipeline and Workflow

4.2 Technical Architecture and Design

4.3 Evaluation Task Implementation and Results

- Task 1: Common Task — COCA Preprocessing and Data Pipeline
- Task 2: Specific Task — Coronary Atlas Registration (Project 3)
- Task 3: Reproducibility and Validation Suite (Tests + Notebooks + Artifacts)

4.4 Evaluation Metrics and Validation Strategy

5. Project Execution Plan

5.1 Objectives and Milestones

5.2 Phase-wise Development Plan

5.3 Detailed Week-by-Week Timeline

6. Additional Information

6.1 Why ML4SCI

6.2 Academic Background

6.3 Technical Skills and Relevant Experience

6.4 Open Source Contributions / Preparation

6.5 Commitment and Availability

7. References

1. Applicant Information

Personal Details

- Name: Karthik
- Email: ``<ADD_EMAIL_HERE>``
- Contact Number: ``<ADD_CONTACT_NUMBER_HERE>``
- University: ``<ADD_UNIVERSITY_NAME_HERE>``
- Degree Program: B.Tech / B.E. in Artificial Intelligence and Machine Learning
- CGPA: ``<ADD_CGPA_HERE>``
- Timezone: IST (UTC+5:30)

Technical Profiles

- GitHub: ``<ADD_GITHUB_PROFILE_LINK_HERE>``
- LinkedIn: ``<ADD_LINKEDIN_LINK_HERE>``
- Portfolio / Personal Website: ``<ADD_PORTFOLIO_LINK_HERE>``

...

2. Abstract

This proposal targets the PrediCT project track on building and comparing strategies for Coronary Artery Calcium (CAC) analysis from non-contrast CT. The core objective is to deliver a reproducible and practical research pipeline that supports preprocessing, registration-guided validation, quantitative evaluation, and clinical-score-aware analysis.

I am submitting this proposal with a working implementation foundation. The repository already contains modular pipeline code, runnable scripts, validation notebooks, tests, metrics outputs, and visualization artifacts. This puts the project in a strong position: the proposal is not only conceptual, but also grounded in executed results.

The planned GSoC work focuses on completing the required segmentation-comparison and evaluation layers while preserving software quality and reproducibility. The final outcome will be a documented, test-backed, and mentor-verifiable framework suitable for future expansion on unlabeled datasets and downstream clinical modeling.

...

3. Problem Context and Motivation

3.1 Background of the Problem

CAC segmentation and quantification are difficult because lesions are usually small, scattered, and sparse relative to total image volume. This creates heavy class imbalance, where models can appear stable while still missing clinically relevant plaques. False positives from nearby bony structures and noise-related

artifacts further complicate model behavior.

Even with good model design, inconsistent preprocessing and weak validation protocols often lead to unstable results across runs or datasets. For a project like PrediCT, where scientific reproducibility is part of the contribution quality, this inconsistency is a major bottleneck.

3.2 Importance in Scientific / ML4SCI Context

In ML4SCI, the contribution is expected to be both technically sound and scientifically reusable. That means the work should not stop at “a model trains”; it should include transparent preprocessing assumptions, reproducible experiments, and interpretable outputs that other researchers can verify.

This proposal aligns with that expectation by:

- integrating deterministic, script-driven workflows,
- preserving measurable validation outputs,
- connecting technical quality to clinically meaningful signals (e.g., calcium proximity and scoring relevance),
- and preparing the codebase for fair model-to-model comparisons.

4. Proposed Solution and Technical Approach

4.1 System Pipeline and Workflow

The proposed system follows a staged workflow:

1. Data ingestion and metadata creation.
2. CT preprocessing (HU clipping, normalization, resampling, region-focused preparation).
3. Registration of atlas information to scan space.
4. Centerline-aware proximity evaluation and result visualization.
5. Experiment reporting through JSON/CSV artifacts and notebooks.

This workflow is designed to support both scientific analysis and engineering reliability. Each stage has explicit inputs, outputs, and run scripts, which reduces ambiguity during review and debugging.

4.2 Technical Architecture and Design

The implementation is organized under a modular package layout:

- ``src/predict_cac/data`` for ingestion and metadata logic,
- ``src/predict_cac/transforms`` for preprocessing operations,
- ``src/predict_cac/registration`` for rigid/affine/deformable registration logic,
- ``src/predict_cac/evaluation`` for proximity metrics and visualization,
- ``src/predict_cac/scoring`` for Agatston-oriented scoring utilities,
- ``src/predict_cac/utils`` for synthetic data fallback and helper functionality.

Primary execution scripts:

- ``scripts/run_ingest.py``
- ``scripts/run_preprocessing.py``
- ``scripts/run_registration.py``
- ``scripts/run_validation.py``
- ``scripts/generate_demo_data.py``

This architecture keeps the pipeline extensible while retaining a clear execution path for mentors and contributors.

4.3 Evaluation Task Implementation and Results

Task 1: Common Task — COCA Preprocessing and Data Pipeline

Implemented:

- HU clipping / windowing tailored for cardiac CT,

- z-score normalization,
- isotropic resampling support,
- efficient dataloader infrastructure,
- metadata generation and dataset-statistics readiness.

Executed evidence (`.venv`):

- `python scripts/run\_ingest.py`
- `python scripts/run\_preprocessing.py`

Observed output summary:

- metadata file generated in `data/metadata.csv`,
- preprocessing executed successfully on available scans.

Task 2: Specific Task — Coronary Atlas Registration (Project 3)

Implemented:

- rigid + affine registration pipeline,
- optional deformable registration hook,
- runtime recording for speed/accuracy trade-off,
- centerline proximity validation and visual checks.

Executed evidence (`.venv`):

- `python scripts/run\_registration.py`
- `python scripts/run\_validation.py`

Latest measured results from current metrics files:

- `mean\_distance\_mm`: **5.0388**
- `median\_distance\_mm`: **2.4495**
- `percent\_within\_10mm`: **79.3217%**
- `runtime\_seconds`: **2.3034**

These results satisfy the PrediCT Project 3 validation target (>70% calcium proximity within ± 10 mm of transformed centerlines).

Task 3: Reproducibility and Validation Suite (Tests + Notebooks + Artifacts)

Implemented and executed:

- notebook-based validation/visualization flow,
- structured artifact persistence,
- focused automated tests for preprocessing, registration metrics, synthetic data, and E2E behavior.

Executed evidence (``.venv``):

- ``pytest tests/test_preprocessing_extended.py tests/test_registration_metrics_extended.py tests/test_synthetic_data.py tests/test_pipeline_e2e.py -q``

Observed output summary:

- ``12 passed`` in focused suite execution.

Artifacts generated and maintained:

- ``experiments/metrics.json``
- ``outputs/validation_metrics.json``
- ``experiments/multi_scan_validation.csv``
- ``outputs/plots/ct_calcium_overlay.png``
- ``outputs/plots/centerline_overlay.png``
- ``outputs/plots/distance_histogram.png``

4.4 Evaluation Metrics and Validation Strategy

Validation is based on both quantitative and qualitative checks.

Quantitative metrics:

- mean calcium-to-centerline distance (mm),
- median calcium-to-centerline distance (mm),
- percent of calcium voxels within threshold (10 mm),
- runtime per scan.

Qualitative metrics:

- CT + calcium overlay,
- CT + centerline overlay,
- distance histogram for distribution-level sanity checks.

Strategy:

- run deterministic scripts through ``.venv``,
- persist metrics and plots in fixed locations,
- cross-check single-run and multi-run outputs,
- use tests and notebooks as independent verification layers.

5. Project Execution Plan

5.1 Objectives and Milestones

Primary objectives:

1. deliver a reproducible preprocessing and data pipeline,
2. finalize robust registration-validation path aligned with test requirements,
3. integrate and compare segmentation strategies under a fair evaluation protocol,
4. report clinically relevant score behavior in a transparent way,
5. provide contributor-ready documentation and maintainability support.

Milestones:

- M1: baseline stabilization and reproducibility lock,
- M2: validated registration + evaluation package,
- M3: segmentation comparison framework and reporting,
- M4: final QA, docs, and handoff.

5.2 Phase-wise Development Plan

Phase A — Baseline and reproducibility hardening

Phase B — Segmentation-comparison setup and baseline runs

Phase C — Clinical-score and result interpretation improvements

Phase D — Testing, notebook reliability, and documentation polish

Phase E — Final review, cleanup, and mentor handoff

5.3 Detailed Week-by-Week Timeline

Community Bonding:

- confirm scope boundaries, communication cadence, and acceptance criteria.

Week 1:

- baseline freeze, command audit, and artifact consistency checks.

Week 2:

- strengthen preprocessing and metadata validation edge cases.

Week 3:

- segmentation-comparison scaffolding and training/evaluation templates.

Week 4:

- baseline model run protocol and reproducible experiment logging.

Week 5:

- registration/evaluation integration into model comparison flow.

Week 6:

- midterm checkpoint: reproducible report package and mentor review.

Week 7:

- improve score-aware comparison analysis and result interpretation.

Week 8:

- notebook hardening and additional validation scenario checks.

Week 9:

- regression checks for outputs, performance, and reproducibility.

Week 10:

- finalize model-comparison summary and quality-control review.

Week 11:

- documentation finalization and onboarding-focused cleanup.

Week 12:

- final QA, submission packaging, and maintainership handoff notes.

6. Additional Information

6.1 Why ML4SCI

ML4SCI offers a meaningful intersection of open-source engineering and high-impact scientific work. The PrediCT project is exactly the type of environment where disciplined pipeline design, reproducibility, and measurable evaluation are as important as model experimentation.

6.2 Academic Background

I am currently a second-year AIML undergraduate, with active focus on machine learning, applied data systems, and practical model evaluation workflows.

6.3 Technical Skills and Relevant Experience

- Python-based ML and pipeline engineering,
- PyTorch workflow familiarity,
- medical-image format handling and preprocessing,
- script and notebook-based experiment execution,
- testing and reproducibility-oriented development.

6.4 Open Source Contributions / Preparation

I have already prepared substantial project-specific work in this repository:

- modular pipeline implementation,
- reproducible script execution path,
- registration-validation outputs,
- test suite expansion,
- notebook run and output verification.

This preparation substantially lowers execution risk for the GSoC development window.

6.5 Commitment and Availability

I can commit consistent weekly effort throughout the coding period and will maintain regular progress updates. I will follow the ML4SCI communication and submission process as instructed (including use of official channels and form-based submission requirements).

...

7. References

- PrediCT Project Page (ML4SCI): https://ml4sci.org/gsoc/projects/2026/project_PREDICT.html
- PrediCT Organizations:

- Alabama: <https://ml4sci.org/gsoc/organizations/2026/alabama.html>
- Kettering: <https://ml4sci.org/gsoc/organizations/2026/kettering.html>
- nnU-Net Repository: <https://github.com/MIC-DKFZ/nnUNet>
- MONAI: <https://github.com/Project-MONAI/MONAI>
- pydicom Documentation: <https://pydicom.github.io/pydicom/stable/>
- SimpleITK Registration Overview: <https://simpleitk.readthedocs.io/en/v2.5.0/registrationOverview.html#registration-overview>
- ImageCAS (Kaggle): <https://www.kaggle.com/datasets/andrewmvd/imagecas>
- Elastix: <https://elastix.dev/>
- ANTs: <https://github.com/ANTsX/ANTs>
- U-Net Paper: <https://arxiv.org/abs/1505.04597>
- Additional reference paper: <https://www.sciencedirect.com/science/article/pii/S136184152500369X>
- Additional reference paper: <https://arxiv.org/abs/2109.03201>
- Additional reference paper: <https://arxiv.org/abs/1612.08894>
- Additional reference paper: <https://www.sciencedirect.com/science/article/pii/S2950162825000025>

...

Repository Links (Fill After Git Push)

- Main Repository: ``<ADD_GITHUB_REPO_LINK_HERE>``
- Proposal Branch: ``<ADD_BRANCH_LINK_HERE>``
- Test Task Code Link: ``<ADD_TEST_TASK_CODE_LINK_HERE>``
- README: ``<ADD_README_LINK_HERE>``
- Source (`src/predict_cac`): ``<ADD_SRC_LINK_HERE>``
- Scripts: ``<ADD_SCRIPTS_LINK_HERE>``
- Tests: ``<ADD_TESTS_LINK_HERE>``
- Notebooks: ``<ADD_NOTEBOOKS_LINK_HERE>``
- Metrics JSON: ``<ADD_METRICS_JSON_LINK_HERE>``
- Multi-scan CSV: ``<ADD_MULTI_SCAN_CSV_LINK_HERE>``
- Plots: ``<ADD_PLOTS_LINK_HERE>``